

RELATIONAL ALGEBRA

The data in relational tables are of limited value unless the data can be manipulated to generate useful information. This section describes the basic data manipulation capabilities of the relational model. **Relational algebra** defines the theoretical way of manipulating table contents using relational operators.

The good news is that as database professionals, we do not have to write mathematical formulas to work with our data. Data is manipulated by database developers and programmers using powerful languages like SQL that hide the underlying math. However, understanding the underlying principles can give you a good feeling for the types of operations that can be performed, and help you to understand how to write your queries more efficiently and effectively.

One advantage of using formal mathematical representations of operations is that mathematical statements are unambiguous. These statements are very specific, and they require that database designers be specific in the language used to explain them. As previously explained, it is common to use the terms *relation* and *table* interchangeably.

One important aspect of using the specific term, *relation*, is that it acknowledges the distinction between the relation and the relation variable, or *relvar*, for short. A relation is the data that we see in our tables. A **relvar** is a variable that holds a relation. For example, imagine you were writing a program and created a variable named *qty* for holding integer data. The variable *qty* is not an integer itself; it is a container for holding integers. Similarly, when you create a table, the table structure holds the table data. The structure is properly called a relvar, and the data in the structure would be a relation. The relvar is a container (variable) for holding relation data, not the relation itself. The data in the table are a relation.

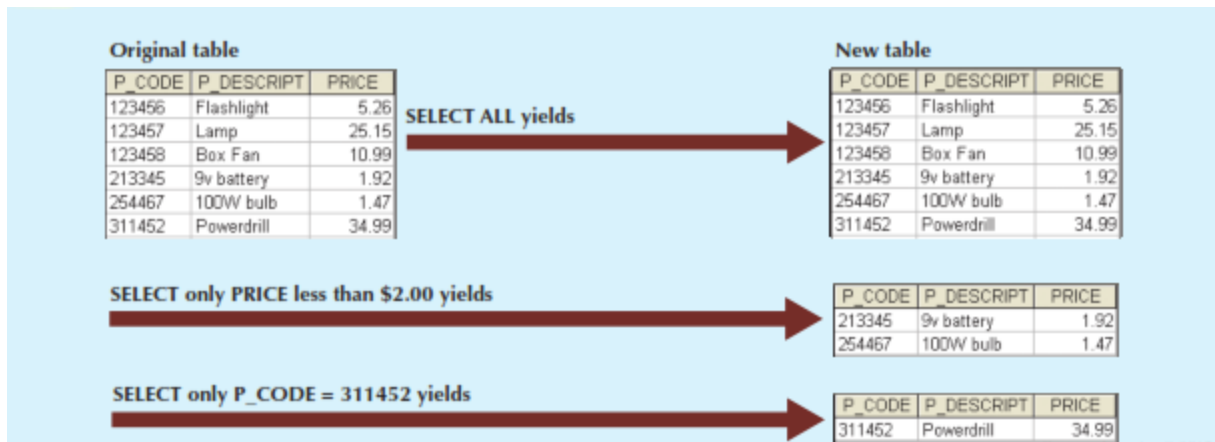
A relvar has two parts: the heading and the body. The relvar heading contains the names of the attributes, while the relvar body contains the relation. To conveniently maintain this distinction in formulas, an unspecified relation is often assigned a lowercase letter (e.g., “*r*”), while the relvar is assigned an uppercase letter (e.g., “*R*”). We could then say that *r* is a relation of type *R*, or *r*(*R*).

RELATIONAL SET OPERATORS

The relational operators have the property of **closure**; that is, the use of relational algebra operators on existing relations (tables) produces new relations. Numerous operators have been defined. Some operators are fundamental, while others are convenient but can be derived using the fundamental operators. In this section, the focus will be on the SELECT (or RESTRICT), PROJECT, UNION, INTERSECT, DIFFERENCE, PRODUCT, JOIN, and DIVIDE operators.

SELECT

SELECT, also known as RESTRICT, is referred to as a unary operator because it only uses one table as input. It yields values for all rows found in the table that satisfy a given condition. SELECT can be used to list all of the rows, or it can yield only rows that match a specified criterion. In other words, SELECT yields a horizontal subset of a table. SELECT will not limit the attributes returned so all attributes of the table will be included in the result. The effect of a SELECT operation is shown in the Figure below.

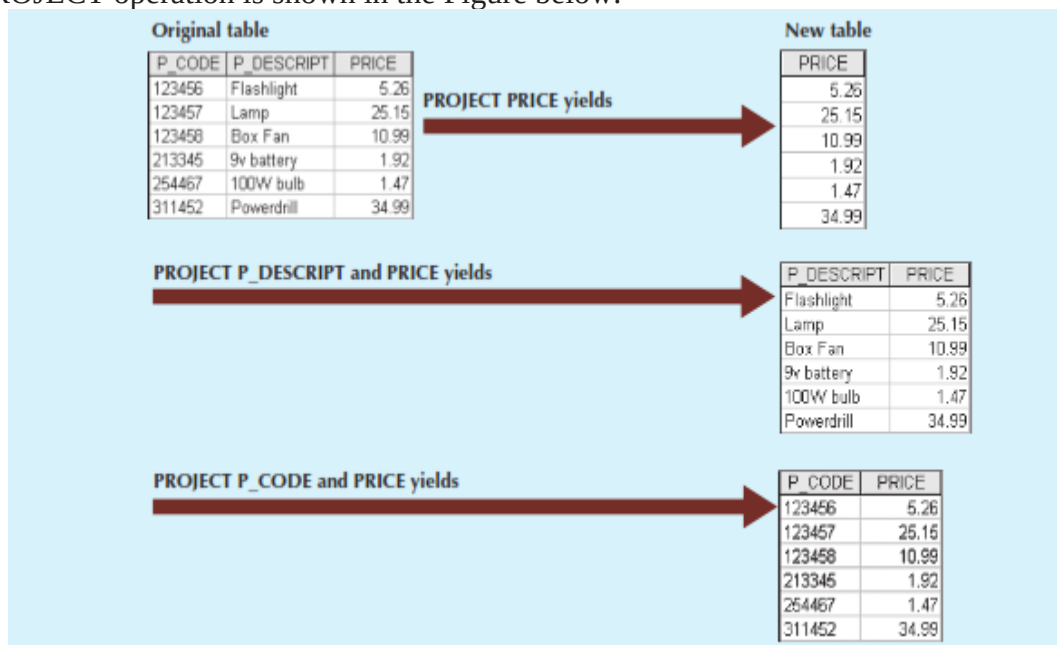


Formally, SELECT is denoted by the lowercase Greek letter sigma (σ). Sigma is followed by the condition to be evaluated (called a predicate) as a subscript, and then the relation is listed in parentheses. For example, to SELECT all of the rows in the CUSTOMER table that have the value '10010' in the CUS_CODE attribute, you would write the following:

$$\sigma_{\text{cus_code} = 10010} (\text{CUSTOMER})$$

PROJECT

PROJECT yields all values for selected attributes. It is also a unary operator, accepting only one table as input. PROJECT will return only the attributes requested, in the order in which they are requested. In other words, PROJECT yields a vertical subset of a table. PROJECT will not limit the rows returned so all rows of the specified attributes will be included in the result. The effect of a PROJECT operation is shown in the Figure below.



Formally, PROJECT is denoted by the Greek letter pi (π). Some sources use the uppercase letter, and other sources use the lowercase letter. Codd used the lowercase p in his original article on the relational model, and that is what we use here. Pi is followed by the list of attributes to be returned as subscripts, and then the relation listed in parentheses. For example, to PROJECT the CUS_FNAME and CUS_LNAME attributes in the CUSTOMER table, you would write the following:

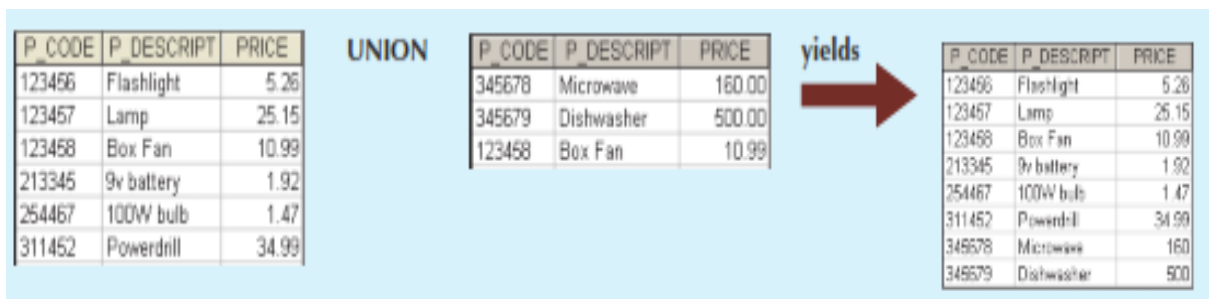
$$\pi_{\text{cus_fname, cus_lname}} (\text{CUSTOMER})$$

Since relational operators have the property of closure, that is, they accept relations as input and produce relations as output, it is possible to combine operators. For example, you can combine the two previous operators to find the customer first and last name of the customer with customer code 10010:

$$\pi_{\text{cus_fname, cus_lname}} (\sigma_{\text{cus_code} = 10010} (\text{CUSTOMER}))$$

UNION (U)

UNION combines all rows from two tables, excluding duplicate rows. To be used in the UNION, the tables must have the same attribute characteristics; in other words, the columns and domains must be compatible. When two or more tables share the same number of columns, and when their corresponding columns share the same or compatible domains, they are said to be **union-compatible**. The effect of a UNION operation is shown in the Figure below.



UNION is denoted by the symbol U. If the relations SUPPLIER and VENDOR are union-compatible, then a UNION between them would be denoted as follows:

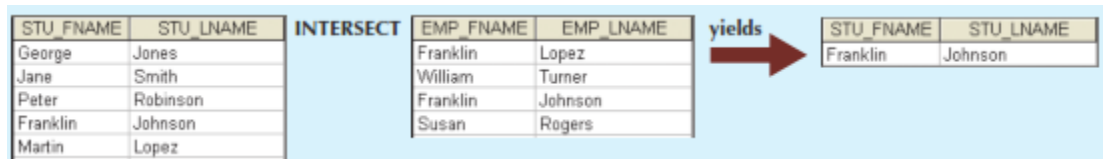
$$\text{supplier} \cup \text{vendor}$$

It is rather unusual to find two relations that are union-compatible in a database. Typically, PROJECT operators are applied to relations to produce results that are union-compatible. For example, assume the SUPPLIER and VENDOR tables are not union-compatible. If you wish to produce a listing of all vendor and supplier names, then you can PROJECT the names from each table and then perform a UNION with them.

$$\pi_{\text{supplier_name}} (\text{supplier}) \cup \pi_{\text{vendor_name}} (\text{vendor})$$

INTERSECT ($\cap$)

INTERSECT yields only the rows that appear in both tables. As with UNION, the tables must be union-compatible to yield valid results. For example, you cannot use INTERSECT if one of the attributes is numeric and one is character based. For the rows to be considered the same in both tables and appear in the result of the INTERSECT, the entire rows must be exact duplicates. The effect of an INTERSECT operation is shown in the Figure below



INTERSECT is denoted by the symbol $\cap$. If the relations SUPPLIER and VENDOR are union-compatible, then an INTERSECT between them would be denoted as follows:

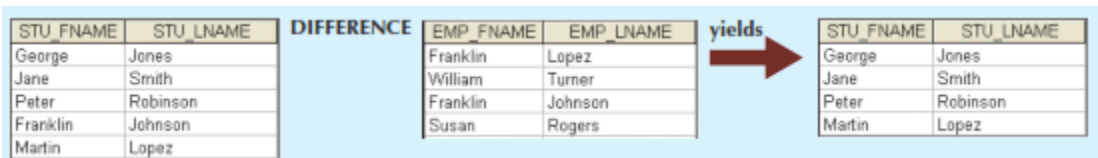
$\text{Supplier} \cap \text{vendor}$

Just as with the UNION operator, it is unusual to find two relations that are union-compatible in a database so PROJECT operators are applied to relations to produce results that can be manipulated with an INTERSECT operator. For example, again assume the SUPPLIER and VENDOR tables are not union-compatible. If you wish to produce a listing of any vendor and supplier names that are the same in both tables, then you can PROJECT the names from each table and then perform an INTERSECT with them.

$\pi_{\text{supplier_name}}(\text{supplier}) \cap \pi_{\text{vendor_name}}(\text{vendor})$

DIFFERENCE (−)

DIFFERENCE yields all rows in one table that are not found in the other table; that is, it subtracts one table from the other. As with UNION, the tables must be union-compatible to yield valid results. The effect of a DIFFERENCE operation is shown in the Figure below. However, note that subtracting the first table from the second table is not the same as subtracting the second table from the first table.



DIFFERENCE is denoted by the minus symbol $-$. If the relations SUPPLIER and VENDOR are union-compatible, then an DIFFERENCE of SUPPLIER minus VENDOR would be written as follows:

$\text{supplier} - \text{vendor}$

Assuming the SUPPLIER and VENDOR tables are not union-compatible, producing a list of any supplier names that do not appear as vendor names, then you can use a DIFFERENCE operator.

$\pi_{\text{supplier_name}}(\text{supplier}) - \pi_{\text{vendor_name}}(\text{vendor})$

PRODUCT

PRODUCT yields all possible pairs of rows from two tables—also known as the Cartesian product. Therefore, if one table has six rows and the other table has three rows, the PRODUCT yields a list composed of $6 * 3 = 18$ rows. The effect of a PRODUCT operation is shown in the Figure below

P_CODE	P_DESCRIPT	PRICE	PRODUCT			yield	P_CODE	P_DESCRIPT	PRICE	STORE	AISLE	SHELF
123456	Flashlight	5.26	23	W	5		123456	Flashlight	5.26	23	W	5
123457	Lamp	25.15	24	K	9		123456	Flashlight	5.26	24	K	9
123458	Box Fan	10.99	25	Z	6		123456	Flashlight	5.26	25	Z	6
213345	9v battery	1.92					123457	Lamp	25.15	23	W	5
254467	100W bulb	1.47					123457	Lamp	25.15	24	K	9
311452	Powerdrill	34.99					123457	Lamp	25.15	25	Z	6
							123458	Box Fan	10.99	23	W	5
							123458	Box Fan	10.99	24	K	9
							123458	Box Fan	10.99	25	Z	6
							213345	9v battery	1.92	23	W	5
							213345	9v battery	1.92	24	K	9
							213345	9v battery	1.92	25	Z	6
							311452	Powerdrill	34.99	23	W	5
							311452	Powerdrill	34.99	24	K	9
							311452	Powerdrill	34.99	25	Z	6
							254467	100W bulb	1.47	23	W	5
							254467	100W bulb	1.47	24	K	9
							254467	100W bulb	1.47	25	Z	6

PRODUCT is denoted by the multiplication symbol X. The PRODUCT of the CUSTOMER and AGENT relations would be written as follows:

Customer X Agent

A Cartesian product produces a set of sequences in which every member of one set is paired with every member of another set. In terms of relations, this means that every tuple in one relation is paired with every tuple in the second relation.

JOIN (

JOIN allows information to be intelligently combined from two or more tables. JOIN is the real power behind the relational database, allowing the use of independent tables linked by common attributes. The CUSTOMER and AGENT tables shown in the Figure below will be used to illustrate several types of joins.

Table name: CUSTOMER				Table name: AGENT	
CUS_CODE	CUS_LNAME	CUS_ZIP	AGENT_CODE	AGENT_CODE	AGENT_PHONE
1132445	Walker	32145	231	125	6152439887
1217782	Adares	32145	125	167	6153426778
1312243	Rakowski	34129	167	231	6152431124
1321242	Rodriguez	37134	125	333	9041234445
1542311	Smithson	37134	421		
1657399	Vanloo	32145	231		

A **natural join** links tables by selecting only the rows with common values in their common attribute(s). A natural join is the result of a three-stage process:

1. First, a **PRODUCT** of the tables is created, yielding the results shown in the Figure below.

CUS_CODE	CUS_LNAME	CUS_ZIP	CUSTOMER.AGENT_CODE	AGENT.AGENT_CODE	AGENT_PHONE
1132445	Walker	32145	231	125	6152439887
1132445	Walker	32145	231	167	6153426778
1132445	Walker	32145	231	231	6152431124
1132445	Walker	32145	231	333	9041234445
1217782	Adares	32145	125	125	6152439887
1217782	Adares	32145	125	167	6153426778
1217782	Adares	32145	125	231	6152431124
1217782	Adares	32145	125	333	9041234445
1312243	Rakowski	34129	167	125	6152439887
1312243	Rakowski	34129	167	167	6153426778
1312243	Rakowski	34129	167	231	6152431124
1312243	Rakowski	34129	167	333	9041234445
1321242	Rodriguez	37134	125	125	6152439887
1321242	Rodriguez	37134	125	167	6153426778
1321242	Rodriguez	37134	125	231	6152431124
1321242	Rodriguez	37134	125	333	9041234445
1542311	Smithson	37134	421	125	6152439887
1542311	Smithson	37134	421	167	6153426778
1542311	Smithson	37134	421	231	6152431124
1542311	Smithson	37134	421	333	9041234445
1657399	Vanloo	32145	231	125	6152439887
1657399	Vanloo	32145	231	167	6153426778
1657399	Vanloo	32145	231	231	6152431124
1657399	Vanloo	32145	231	333	9041234445

2. Second, a **SELECT** is performed on the output of Step 1 to yield only the rows for which the **AGENT_CODE** values are equal. The common columns are referred to as the **join columns**. Step b yields the results shown in the figure below

CUS_CODE	CUS_LNAME	CUS_ZIP	CUSTOMER.AGENT_CODE	AGENT.AGENT_CODE	AGENT_PHONE
1217782	Adares	32145	125	125	6152439887
1321242	Rodriguez	37134	125	125	6152439887
1312243	Rakowski	34129	167	167	6153426778
1132445	Walker	32145	231	231	6152431124
1657399	Vanloo	32145	231	231	6152431124

3. A **PROJECT** is performed on the results of Step 2 to yield a single copy of each attribute, thereby eliminating duplicate columns. Step 3 yields the output shown in the Figure below.

CUS_CODE	CUS_LNAME	CUS_ZIP	AGENT_CODE	AGENT_PHONE
1217782	Adares	32145	125	6152439887
1321242	Rodriguez	37134	125	6152439887
1312243	Rakowski	34129	167	6153426778
1132445	Walker	32145	231	6152431124
1657399	Vanloo	32145	231	6152431124

The final outcome of a natural join yields a table that does not include unmatched pairs and provides only the copies of the matches.

Note a few crucial features of the natural join operation:

- If no match is made between the table rows, the new table does not include the unmatched row. In that case, neither AGENT_CODE 421 nor the customer whose last name is Smithson is included. Smithson's AGENT_CODE 421 does not match any entry in the AGENT table.
- The column on which the join was made—that is, AGENT_CODE—occurs only once in the new table.
- If the same AGENT_CODE were to occur several times in the AGENT table, a customer would be listed for each match. For example, if the AGENT_CODE 167 occurred three times in the AGENT table, the customer named Rakowski would also occur three times in the resulting table because Rakowski is associated with AGENT_CODE 167. (Of course, a good AGENT table cannot yield such a result because it would contain unique primary key values.)

Natural join is normally just referred to as JOIN in formal treatments. JOIN is denoted by the symbol $\bowtie$. The JOIN of the CUSTOMER and AGENT relations would be written as follows:

Customer $\bowtie$ agent